MODULE 14 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

# Workflows and multistep reasoning

The complexity ladder, pipelines, concurrent flows, reflection loops, chain-of-thought evaluation, and hybrid model and rules designs.

DOMAIN 2 - Generative AI and agents  WEIGHT 30-35%  MODULE 14 of 39

admin@sthithapragna.com

Use the **least intelligence** that reliably does the job. Code where you can, a model where you must, and loops only with a limit.

# Start with the right level of complexity

| LEVEL | USE IT WHEN | WATCH OUT FOR |
|---|---|---|
| Direct model call | One pass does it: classify, summarise, translate | Nothing extra. The simplest option |
| Single agent with tools | Varied requests in one domain need tool calls | Set iteration limits on tool loops |
| Multi-agent orchestration | Cross-domain work, security boundaries, parallel specialists | Overhead, latency, new failure modes |

ON THE EXAM

Microsoft: use the lowest level of complexity that reliably meets your requirements.

If a prompt can solve it, you don't need an agent. If a rule can decide it, you don't need a model.


admin@sthithapragna.com

# The tool loop, and how to keep it safe

- The model **requests** a tool call; your code or the platform **runs** it; the result goes back.
- The model may call several tools in turn before it answers.
- **Set an iteration limit**. Without one, a confused model can loop through tools indefinitely.
- **Validate tool results** and surface errors, instead of passing failures silently back.
- Treat tool output as untrusted input, as module 09 showed.

> ON THE EXAM
>
> A tool loop is a reasoning pipeline the model steers. Its safety comes from limits and validation in your code.

# Four ways to structure multistep work

| PATTERN | HOW IT WORKS | AVOID WHEN |
|---|---|---|
| Sequential | Each step builds on the last | Steps are independent |
| Concurrent | Independent steps run in parallel | Order or shared state matters |
| Deterministic routing | Classify, then send down a path | The route emerges mid-task |
| Maker-checker loop | Draft, critique, revise | No clear acceptance criteria |

ON THE EXAM

Combine shapes per stage. A pipeline can run sequentially, fan out for analysis, then loop for review.

# Fan out, then bring it back together

- Run **independent** steps on the same input at the same time, to cut total latency.
- **Aggregate** by voting, by weighted merging, or with a model that summarises the results.
- Decide in advance how to resolve **conflicting** results.
- Parallel calls hit **quota** at the same moment, so check rate limits first.
- Don't let parallel steps update the same shared state.

> THE SIGNAL
>
> Two slow steps that don't depend on each other are the signal for fan-out and fan-in.

Validate between steps. A **malformed output** at step one becomes a **confident wrong answer** at step four.


admin@sthithapragna.com

# Microsoft Agent Framework workflows

- **Explicit, inspectable** paths that coordinate code, agents, state and human input.
- **Executors** are the steps: plain functions, or agents. **Edges** connect them.
- Edges can be direct, **conditional**, **fan-out** or **fan-in**.
- **Checkpoints** let long workflows resume. **Human-in-the-loop** requests pause and wait.
- Two styles: a graph built with WorkflowBuilder, or plain Python functions (experimental).

> ON THE EXAM
>
> A workflow puts the control flow in code you can read. An agent decides its own next step.

# What to build workflows with now

RETIRING

## Don't start new work here

- Prompt flow: retires 20 April 2027
- Foundry workflows: retire 1 December 2026
- Older answers may still name them

CURRENT

## Build here

- Microsoft Agent Framework, GA April 2026
- Workflows plus multi-agent orchestrations
- Your own code, for simple pipelines

ON THE EXAM

If an older question's answer is prompt flow, the current equivalent is an Agent Framework workflow. Module 33 maps these.

# Maker-checker loops

- A **maker** drafts. A **checker** evaluates the draft against defined criteria.
- If it fails, feedback goes back and the maker revises. Repeat until it passes.
- Also called **reflection loops**, evaluator-optimizer, or generator-verifier.
- Needs **clear acceptance criteria**, or the checker can't decide consistently.
- Needs an **iteration cap** and a **fallback**: escalate to a human, or return the best draft with a warning.

> THE TIEBREAKER
>
> A reflection loop without a cap is an unbounded cost. Microsoft's guidance requires all three: criteria, cap, fallback.

# A reflection loop with a cap and a fallback

```
draft = client.responses.create(model="writer", input=brief).output_text
for attempt in range(3):
    review = client.responses.create(model="reviewer",
        instructions="Judge the draft against the criteria.",
        input=f"Criteria:\n{criteria}\n\nDraft:\n{draft}",
        text={"format": {"type": "json_schema", "name": "review",
                         "strict": True, "schema": review_schema}})
    verdict = json.loads(review.output_text)
    if verdict["pass"]:
        break
    draft = client.responses.create(model="writer",
        input=f"Revise using this feedback:\n{verdict['feedback']}\n\n{draft}").output_text
else:
    escalate_to_human(draft, verdict["feedback"])
```

# Evaluating the reasoning, not just the answer

- A right answer from wrong reasoning will fail on the next case. So **evaluate the steps**.
- With standard models, ask for reasoning in **separate structured fields**, then score them with a rubric.
- Reasoning models **don't return raw reasoning**. You get a **reasoning summary** if you request one.
- Trying to extract raw reasoning another way is unsupported, and may breach the usage policy.
- For agents, evaluators score intermediate behaviour: **intent resolution, tool call accuracy, task adherence**.

> ON THE EXAM
>
> For reasoning models, evaluate what you can observe: summaries, tool calls, and the final output. Module 15 goes deeper.

# Ways to combine models in one flow

| APPROACH | HOW IT WORKS | GOOD FOR |
|---|---|---|
| Model router | Platform picks a model per prompt | Mixed traffic, one deployment |
| A model per step | Small models extract; large ones synthesise | Cheaper pipelines |
| Escalation | Small model first; hard cases go larger | Mostly easy traffic |
| Foundry Tools as steps | Translator, Speech, Content Understanding | Narrow, purpose-built jobs |

ON THE EXAM

Microsoft: give each step a model matched to its complexity.

# What should the model decide, and what should rules?

THE MODEL

## Language and judgement

- Understand free-text requests
- Extract fields from messy input
- Draft, summarise and explain

RULES IN CODE

## Fixed decisions

- Thresholds and limits
- Eligibility and compliance gates
- Calculations and fixed routing

THE TIEBREAKER

The strongest design: the model extracts, the rules decide, the model explains the decision in plain language.


admin@sthithapragna.com

# A checklist for multistep flows

| PRACTICE | PREVENTS |
| --- | --- |
| Iteration limits on every loop | Runaway tool calls and endless revision |
| Validate each step's output | Bad output cascading through the pipeline |
| Timeouts, retries, surfaced errors | Silent hangs and hidden failures |
| Checkpoints on long workflows | Losing all progress after an interruption |
| Right-sized model per step | Paying frontier prices for simple steps |
| Compact context between steps | Context windows growing until quality drops |

ON THE EXAM

Every row is from Microsoft's orchestration guidance. Wrong answers usually omit the limit or the validation.

> A helpdesk wants to sort incoming support emails into one of six fixed categories, then place each email in the matching team's queue.
>
> An architect proposes a triage agent that hands each email off to one of six specialist agents.

## What should the team build instead?

A One model call that returns a category; code routes the email.

B A handoff orchestration with a triage agent and six specialist agents.

C A Magentic manager that plans how each email should be routed.

D A group chat in which the six specialist agents vote on each email.

◆ PAUSE AND DECIDE

## What should the team build instead?

**A** One model call, then code routes.

Classification is a single-pass task. The route follows from the category, so code decides it.

**B** Handoff orchestration.

*Microsoft: avoid handoff when the right destination is known from the initial input.*

**C** Magentic manager.

*Built for open-ended problems with no known plan. Sorting into six queues has one.*

**D** Group chat vote.

*Seven model calls and a debate for a job one call does. Overhead with no benefit.*

THE SIGNAL

Fixed categories, fixed destinations: the lowest rung of the ladder, plus deterministic routing.

> A marketing team uses a writer model and a reviewer model in a loop to produce product copy. The reviewer is told to 'make sure it is really good'.
>
> Some requests loop more than forty times without finishing, and costs have spiked.

## What should the team change?

A Add two more reviewer models so the loop gets a wider range of opinions.

B Give the reviewer explicit criteria, cap the loop, escalate at cap.

C Remove the reviewer and publish whatever the writer's first draft says.

D Raise the writer model's reasoning effort so drafts improve more quickly.

◆ PAUSE AND DECIDE

# What should the team change?

A More reviewers.

*More opinions with no criteria means more disagreement and even longer loops.*

B Criteria, cap, escalation.

Consistent pass decisions, bounded cost, and a defined outcome when the cap is reached.

C Remove the reviewer.

*Stops the loop by abandoning quality control entirely.*

D More reasoning effort.

*Better drafts can't satisfy a checker with no definition of 'good'.*

THE SIGNAL

'Really good' isn't an acceptance criterion. Criteria, cap and fallback are all three required.

An insurer's assistant reads claim emails, extracts the details, and drafts a decision letter. Policy states that any claim over 10,000 pounds, or on a lapsed policy, must go to a human.

The rule is written in the system prompt, but the assistant occasionally approves claims on lapsed policies.

## What should the team do?

A Rewrite the rule in capitals and repeat it at the end of the system prompt.

B Switch to a reasoning model with high effort so it applies the rule better.

C Add several worked examples of lapsed policies to the system prompt.

D Let the model extract the facts, and apply the rule in code afterwards.

◆ PAUSE AND DECIDE

## What should the team do?

A Capitals and repetition.

*Improves the odds, but 'occasionally' remains. A prompt can't guarantee a rule.*

B Reasoning model, high effort.

*More reliable and much more expensive, but still probabilistic for a fixed rule.*

C Few-shot examples.

*Helps recognition, but the decision is still left to the model.*

D Model extracts; code decides.

A threshold and a status check are fixed rules. Code applies them every time.

THE TIEBREAKER

A compliance rule with a number in it belongs in code. The model's job is extracting the number accurately.

A due-diligence pipeline runs four steps in order: extract company data, analyse financials, assess legal risk, then write a report. The two analysis steps don't depend on each other, and each takes several minutes.

The extraction step sometimes returns malformed data, and the final reports then contain confident errors.

## What should the team change?

A Replace the pipeline with a Magentic manager that plans each run.

B Merge all four steps into one very long prompt to one large model.

C Validate the extraction, then run both analyses in parallel.

D Add a group chat step where all four agents debate the final report.

◆ PAUSE AND DECIDE

# What should the team change?

A Magentic manager.

*For open-ended problems. This pipeline's plan is already known and fixed.*

B One long prompt.

*Loses the step structure, and malformed extraction still flows into the report.*

C Validate, then fan out and fan in.

Stops bad data at step one, and runs the independent slow steps together.

D Group chat debate.

*Adds cost and latency at the end, after bad data has already spread.*

THE SIGNAL

Two problems, two fixes: early failures propagate, so validate; independent slow steps, so run them concurrently.

# Workflows and reasoning in five lines

- **Least intelligence that works:** prompt before agent, rules before model.
- **Sequential** for dependencies, **concurrent** for independent steps, **validate** between them.
- **Reflection loops need criteria, a cap, and a fallback.** All three.
- **Prompt flow is retiring.** Build workflows with Microsoft Agent Framework.
- **Hybrid:** the model extracts, rules decide, the model explains.

> ON THE EXAM
>
> Next, module 15: evaluating models and apps, including fabrication, relevance, quality and safety.